

```
<!--Sistemas operativos-->
```

Modulo de Kernel

```
<Por="Samuel Gomez"/>
```

```
<Por="Nicolas Nevares"/>
```

```
}
```



¿Que es? {

¿Qué es un módulo de kernel?

- Es código que se carga y descarga dinámicamente en el kernel de Linux.
- Permite agregar funcionalidad sin recompilar todo el kernel.
- Se usa para drivers, sistemas de archivos, dispositivos de caracteres, etc.
- Trabaja en espacio de kernel, con más privilegios que un programa normal.

Imagina un coche al que le puedes poner o quitar un portabicicletas, una parrilla, una bocina extra, etc.

El coche sigue siendo el mismo, solo que a veces tiene el accesorio y a veces no.

El módulo de kernel es ese “accesorio” que le conectas al sistema operativo.

}

¿Para qué hacer un módulo de kernel? {

El objetivo de este proyecto es practicar cómo interactúa el kernel con el espacio de usuario, usando un módulo propio. En lugar de solo ejecutar programas normales, trabajamos con un dispositivo en /dev que está conectado a nuestro módulo.

Esto nos permite entender mejor cómo se manejan los drivers, cómo se registran mensajes en la bitácora (dmesg) y cómo se implementan lecturas y escrituras desde el kernel.

}

Explicacion del
codigo {

<"examen final"/>

}

Cabeceras {

- `#include <linux/module.h>`
 - Permite crear módulos de kernel (`module_init`, `module_exit`, `MODULE_LICENSE`, etc.).
- `#include <linux/fs.h>`
 - Cosas de filesystem: estructura `file_operations`, tipos de archivo, permisos, etc.
- `#include <linux/kernel.h>`
 - Funciones generales del kernel: `printk`, macros básicas, tipos comunes.
- `#include <linux/cdev.h>`
 - Soporte para dispositivos de caracteres (`struct cdev`, `cdev_init`, `cdev_add`).
- `#include <linux/uaccess.h>`
 - Funciones para copiar datos entre kernel y user space: `copy_to_user`, `copy_from_user`.
- `#include <linux/device.h>`
 - Manejo de dispositivos y clases en `/sys` y `/dev`: `class_create`, `device_create`, etc.
- `#include <linux/kdev_t.h>`
 - Macros para trabajar con números de dispositivo: `MKDEV`, `MAJOR`, `MINOR`.
- `#include <linux/slab.h>`
 - Memoria dinámica en el kernel: `kmalloc`, `kfree` (tipo `malloc` pero versión kernel).
- `#include <linux/err.h>`
 - Macros para manejar punteros de error: `IS_ERR`, `PTR_ERR`, etc.

}

Estructura {

```
// Metadatos del Módulo  
MODULE_LICENSE("GPL");  
MODULE_DESCRIPTION("Modulo kerneles");
```

```
MODULE_LICENSE("GPL");
```

Qué es: Le dice al kernel con qué licencia está hecho tu módulo.
Por qué importa:

Si es GPL, el kernel lo considera compatible.
No marca el sistema como "tainted".
Te deja usar ciertas funciones internas reservadas para código GPL.

```
MODULE_DESCRIPTION("Modulo kerneles");
```

Qué es: Un texto corto que describe el módulo.
Para qué sirve:

Sale cuando haces `modinfo nombre_modulo`.
Sirve para que cualquiera vea rápido "de qué va" tu módulo sin abrir el código.

}

Estructura {

```
// Constantes
```

```
#define DEVICE_NAME "finalkernel_dev"  
#define CLASS_NAME "finalkernel_class"  
#define BUFFER_SIZE 1024
```

- **DEVICE_NAME**: nombre del archivo que aparecerá en /dev
- → Ej: /dev/finalkernel_dev.
- **CLASS_NAME**: nombre de la "clase" del dispositivo en sysfs
- → Sirve para que el kernel agrupe tu dispositivo.
- **BUFFER_SIZE**: tamaño máximo (en bytes) de los buffers que usas para mensajes

- **dev_major**: guarda el número mayor que te asigna el kernel para el dispositivo.
- **exam_class**: apunta a la clase que creas con class_create (la "categoría" del dispositivo).
- **exam_cdev**: es la estructura del dispositivo de caracteres que registras con cdev_add.

```
// Variables Globales
```

```
static int dev_major = 0;  
static struct class *exam_class = NULL;  
static struct cdev exam_cdev;
```

```
// Variables de estado
```

```
static int stream_active = 0;  
static int sent_once = 0;
```

- **stream_active**:
 - 0 → modo normal.
 - 1 → modo flujo (mensaje continuo).
- **sent_once**:
 - Indica si el mensaje ya se envió una vez en modo normal para luego regresar EOF.

```
}
```

Estructura {

```
static char msg_buffer[BUFFER_SIZE] = "Todo funcionando por aqui!!\n";
static char recv_buffer[BUFFER_SIZE];
```

- `msg_buffer`: texto que el módulo manda al usuario cuando haces `cat /dev/finalkernel_dev`.
- `recv_buffer`: donde se guarda lo que el usuario envía con `echo` para luego imprimirlo en la bitácora.

// Prototipos de funciones

```
static int dev_open(struct inode *, struct file *);
static int dev_release(struct inode *, struct file *);
static ssize_t dev_read(struct file *, char __user *, size_t, loff_t *);
static ssize_t dev_write(struct file *, const char __user *, size_t, loff_t *);
```

- `dev_open`
 - Se llama cuando el usuario hace algo que abre el archivo: `cat /dev/finalkernel_dev`
 - Aquí puedes inicializar cosas o solo registrar que se abrió el dispositivo.
- `dev_release`
 - Se llama cuando el archivo se cierra (cuando termina el `cat`, por ejemplo).
 - Sirve para limpiar o registrar que el dispositivo ya no se está usando.
- `dev_read`
 - Se ejecuta cuando el usuario lee del dispositivo (`cat`, `read`).
 - Tú copias datos desde el kernel al usuario (desde `msg_buffer` hacia `buffer`).
- `dev_write`
 - Se ejecuta cuando el usuario escribe al dispositivo (`echo "texto" > /dev/...`).
 - Tú copias datos desde usuario al kernel (`recv_buffer`) y decides qué hacer con la cadena.

}

Estructura {

```
static struct file_operations fops = {  
    .open = dev_open,  
    .read = dev_read,  
    .write = dev_write,  
    .release = dev_release,  
};
```

Este bloque es donde le dices al kernel qué funciones usar cuando alguien interactúa con tu dispositivo.

- `struct file_operations fops`
 - Es una tabla de “callbacks” del dispositivo de caracteres.
- `.open = dev_open`
 - Cuando alguien abre `/dev/finalkernel_dev`, el kernel llama a `dev_open`.
- `.read = dev_read`
 - Cuando alguien lee (`cat`, `read`), usa tu función `dev_read`.
- `.write = dev_write`
 - Cuando alguien escribe (`echo "..."` `>` `/dev/...`), usa `dev_write`.
- `.release = dev_release`
 - Cuando se cierra el archivo, llama a `dev_release`.

```
static int __init final_ops_init(void) {
    int error = 0;
    dev_t dev_num;

    printk(KERN_INFO "FinalKernel: Inicializando con GOTO...\n");

    // 1. Asignar número mayor dinámicamente
    error = alloc_chrdev_region(&dev_num, 0, 1, DEVICE_NAME);
    if (error < 0) {
        printk(KERN_ALERT "Finalkernel: Fallo al asignar numero mayor\n");
        return error;
    }

    dev_major = MAJOR(dev_num);
    printk(KERN_INFO "Finalkernel: Registrado con Major Number: %d\n", dev_major);

    // 2. Crear la clase del dispositivo (sysfs)
    exam_class = class_create(CLASS_NAME);
    if (IS_ERR(exam_class)) {
        printk(KERN_ALERT "Finalkernel: Fallo al crear la clase\n");
        error = PTR_ERR(exam_class);
        goto r_unreg; // <--- SALTO A LIMPIEZA NIVEL 1
    }

    // 3. Crear el nodo del dispositivo (/dev/finalkernel_dev)
    if (device_create(exam_class, NULL, dev_num, NULL, DEVICE_NAME) == NULL) {
        printk(KERN_ALERT "Finalkernel: Fallo al crear el dispositivo\n");
        error = -1;
        goto r_class; // <--- SALTO A LIMPIEZA NIVEL 2
    }

    // 4. Inicializar cdev y añadirlo al kernel
    cdev_init(&exam_cdev, &fops);
    if (cdev_add(&exam_cdev, dev_num, 1) == -1) {
        printk(KERN_ALERT "Finalkernel: Fallo al añadir cdev\n");
        error = -1;
        goto r_device; // <--- SALTO A LIMPIEZA NIVEL 3
    }

    printk(KERN_INFO "Finalkernel: Modulo cargado correctamente. Dispositivo: /dev/%s\n", DEVICE_NAME);
    return 0; // Éxito total
}
```

- **Pide un número de dispositivo (major/minor)**
 - Usa alloc_chrdev_region para registrar un número mayor dinámico.
 - Si falla, lo reporta con printk y regresa el error.
- **Crea la clase del dispositivo**
 - exam_class = class_create(CLASS_NAME);
 - Si falla, guarda el error y salta con goto r_unreg para liberar el número de dispositivo.
- **Crea el nodo en /dev**
 - device_create(...) crea /dev/finalkernel_dev.
 - Si falla, salta a r_class para destruir la clase y luego liberar el número.
- **Inicializa el cdev y lo registra**
 - cdev_init(&exam_cdev, &fops);
 - cdev_add(&exam_cdev, dev_num, 1);
 - Si falla, salta a r_device para borrar el nodo en /dev, destruir la clase y liberar el número.
- **Si todo sale bien**
 - Imprime que el módulo se cargó correctamente y devuelve 0.

```

    r_device:
        device_destroy(exam_class, dev_num);
    r_class:
        class_destroy(exam_class);
    r_unreg:
        unregister_chrdev_region(dev_num, 1);

    return error;
}

```

- **r_device:**
 - Si ya se había creado el nodo /dev/finalkernel_dev, aquí se borra con device_destroy(...).
- **r_class:**
 - Destruye la clase del dispositivo creada con class_create.
- **r_unreg:**
 - Libera el número de dispositivo (major/minor) con unregister_chrdev_region.

- **__exit final_ops_exit**
 - Función que se ejecuta al descargar el módulo.
- **dev_t dev_num = MKDEV(dev_major, 0);**
 - Reconstruye el número de dispositivo (major/minor) que se había usado.
- **cdev_del(&exam_cdev);**
 - Quita el cdev del kernel (deja de atender el dispositivo de caracteres).
- **device_destroy(exam_class, dev_num);**
 - Borra el nodo /dev/finalkernel_dev.
- **class_destroy(exam_class);**
 - Elimina la clase creada para el dispositivo.
- **unregister_chrdev_region(dev_num, 1);**
 - Libera el número mayor/menor que se había reservado.

```

* @brief
*/
static void __exit final_ops_exit(void) {
    dev_t dev_num = MKDEV(dev_major, 0);

    cdev_del(&exam_cdev);
    device_destroy(exam_class, dev_num);
    class_destroy(exam_class);
    unregister_chrdev_region(dev_num, 1);

    printk(KERN_INFO "Finalkernel: Modulo descargado. Hasta luego!\n");
}

```

```
static int dev_open(struct inode *inodep, struct file *filep) {
    printk(KERN_INFO "Finalkernel: Dispositivo abierto\n");
    if (!stream_active) {
        sent_once = 0;
    }
    return 0;
}
```

- Calcula el largo del mensaje (msg_len) que está en msg_buffer.
- Modo normal:
 - Si ya se envió una vez (sent_once == 1) y no está activo el flujo, regresa 0 → el cat termina.
- Si aún puede enviar, usa copy_to_user para copiar el mensaje del kernel al usuario.
- Si la copia fue bien:
 - En modo normal marca sent_once = 1.
 - En modo flujo no lo marca, así puede seguir enviando.
 - Escribe en la bitácora si fue modo normal o flujo y regresa msg_len.
- Si hay error en copy_to_user, registra el fallo y regresa -EFAULT.

- `printk(KERN_INFO "Finalkernel: Dispositivo abierto\n");`
 - Registra en la bitácora que alguien abrió el dispositivo.
- `if (!stream_active) { sent_once = 0; }`
 - Si no estamos en modo flujo, reinicia sent_once a 0.
 - Esto hace que, en modo normal, cada vez que vuelvas a abrir el dispositivo, el mensaje se pueda enviar una vez más.
- `return 0;`
 - Indica que la apertura fue correcta, sin errores.

```
static ssize_t dev_read(struct file *filep, char __user *buffer, size_t len, loff_t *offset) {
    int errors = 0;
    int msg_len = strlen(msg_buffer);

    // Lógica de Modos:
    if (!stream_active && sent_once) {
        return 0; // EOF, detiene el cat
    }

    errors = copy_to_user(buffer, msg_buffer, msg_len);

    if (errors == 0) {
        if (!stream_active) {
            sent_once = 1; // Marcamos como enviado para que la proxima lectura de 0
            printk(KERN_INFO "Finalkernel: Mensaje enviado a usuario (Modo Normal)\n");
        } else {
            printk(KERN_INFO "Finalkernel: Mensaje enviado a usuario (Modo Flujo)\n");
        }
        return msg_len;
    } else {
        printk(KERN_ALERT "Finalkernel: Error enviando datos al usuario\n");
        return -EFAULT;
    }
}
```



```
static ssize_t dev_write(struct file *filep, const char __user *buffer, size_t len, loff_t *offset) {
    if (len > BUFFER_SIZE - 1) len = BUFFER_SIZE - 1;

    if (copy_from_user(recv_buffer, buffer, len) != 0) {
        return -EFAULT;
    }
    recv_buffer[len] = '\0'; // Asegurar terminación de cadena

    // Imprimir en bitácora lo recibido
    printk(KERN_INFO "FINAL_OPS: Recibido desde usuario: %s", recv_buffer);

    // Lógica de control de flujo
    if (recv_buffer[0] == '1') {
        stream_active = 1;
        printk(KERN_INFO "Finalkernel: ==> MODO FLUJO ACTIVADO (CTRL-C para detener) <==\n");
    } else if (recv_buffer[0] == '0') {
        stream_active = 0;
        sent_once = 0;
        printk(KERN_INFO "Finalkernel: ==> MODO FLUJO DESACTIVADO (Default) <==\n");
    } else {
        printk(KERN_INFO "Finalkernel: Comando desconocido. Use '1' para flujo, '0' para normal.\n");
    }

    return len;
}
```

Limita el tamaño de lo que el usuario manda para que quepa en `recv_buffer`. Usa `copy_from_user` para copiar la cadena desde user space al buffer del kernel. Termina la cadena con `'\0'` y la imprime en la bitácora (`printk`). Revisa el primer carácter:

- Si es `'1'` → activa `stream_active = 1` → modo flujo.
- Si es `'0'` → pone `stream_active = 0` y `sent_once = 0` → modo normal.
- Si es otra cosa → solo avisa que el comando es desconocido.

Devuelve `len` para indicar cuántos bytes recibió.

```
static int dev_release(struct inode *inodep, struct file *filep) {  
    printk(KERN_INFO "Finalkernel: Dispositivo cerrado\n");  
    return 0;  
}
```

dev_release(...)

- Se llama cuando se cierra el archivo /dev/finalkernel_dev
- (cuando termina el cat o el proceso deja de usar el dispositivo).
- Solo registra en la bitácora:
- Finalkernel: Dispositivo cerrado.
- Devuelve 0 → no hubo error al cerrar.

```
module_init(final_ops_init);  
module_exit(final_ops_exit);
```

module_init / module_exit

- module_init(final_ops_init);
 - Le dice al kernel: “cuando cargues este módulo, llama a final_ops_init”.
- module_exit(final_ops_exit);
 - Le dice al kernel: “cuando descargues este módulo, llama a final_ops_exit”

Son los “puntos de entrada y salida” del módulo.

```
<!--Estudio Shonos-->
```

Gracias {

}